Follow-up security review — Marketing25 pharmacy poster platform

Date: 3 October 2026 (second review, after the first round of fixes) · Scope: web app (Next.js BFF), .NET API, ai-service (FastAPI), Dockerfiles, Azure Bicep, deployment scripts and configuration · Type: static source, configuration and dependency review

Summary

This second review re-checked the whole project after the first round of fixes and found **16 new issues**. The two that matter most: **real Meta and Google OAuth secrets are committed** in the development settings file (R2-01) and should be rotated today, and the **web app runs Next.js 14, which no longer gets security fixes** (R2-02). After those come a probable header-spoofing gap that lets callers dodge the new rate limits (R2-03), an open redirect on the sign-in page (R2-04), two gaps in the image defences that together may re-open server-side fetching (R2-05), and the lack of consent and unsubscribe handling for patient emails (R2-06).

Severity	New findings	Meaning
High	2	Fix now
Medium	5	Fix in the next cycle
Low	7	Hardening; schedule
Info	2	Process / awareness

Remediation update

Fixes were made the same day. **Fixed:** R2-01, R2-03, R2-04, R2-05, R2-06, R2-08, R2-09, R2-11, R2-12, R2-13, R2-14, R2-15, R2-16. **Partly fixed:** R2-07, R2-10. **Open:** R2-02. R2-02 (Next.js) and the .NET part of R2-07 need packages and an SDK this review could not install; the exact steps are in docs/UPGRADE_RUNBOOK_2026-10.md. R2-10's remaining items are Azure settings. Each finding below ends with its status. New tests: Acme.Tests/Security/SecurityFollowupFixTests, Acme.IntegrationTests/SecurityFollowupTests, the web tests for forwarding and safe-redirect, and test_security_review.py. `pnpm test:prove -- R2` re-introduces each bug. The web proofs were run here and caught; the .NET and Python ones need dotnet and pytest and have not been run yet. New deploy settings are listed in PRODUCTION_CHECKLIST.md §0.

New findings at a glance

ID	Severity	Finding	Status
R2-01	**High**	Meta app secret and Google OAuth client secret committed to git	**Fixed**
R2-02	**High**	Next.js 14 is end-of-life and no longer receives security fixes	**Open**
R2-03	**Medium**	Per-address rate limits and audit IPs trust a header the caller controls	**Fixed**
R2-04	**Medium**	Open redirect after sign-in	**Fixed**
R2-05	**Medium**	Two gaps in the SEC-01/SEC-10 image defences	**Fixed**
R2-06	**Medium**	Patient marketing emails have no consent record or unsubscribe	**Fixed**
R2-07	**Medium**	Runtimes at or near end of support	**Partly fixed**
R2-08	**Low**	The account lock can be used to lock someone else out	**Fixed**
R2-09	**Low**	Access tokens outlive deactivation, password reset and sign-out	**Fixed**
R2-10	**Low**	Azure resources are reachable from the internet and secrets are not in Key Vault	**Partly fixed**
R2-11	**Low**	ai-service /diagnostics/jwt is anonymous in production	**Fixed**
R2-12	**Low**	No size or shape limit on saved posters	**Fixed**

ID	Severity	Finding	Status
R2-13	Low	One secret signs everything	Fixed
R2-14	Low	ImageMagick security policy loosened in the ai-service image	Fixed
R2-15	Info	Personal and possibly health data handling	Fixed
R2-16	Info	Supply-chain and process gaps	Fixed

Recommended order of work (as reviewed)

Today: rotate the Meta app secret and Google client secret (R2-01), and the Mailjet key while you are there (SEC-03).

This sprint: R2-04 (a one-function fix), R2-05 (validate the parsed canvas; refuse nested SVG), R2-03 (restrict API and ai-service ingress to the web app), R2-11.

Next sprint: R2-02 and R2-07 together: Next 15/16 on Node 22+, then .NET 10 before 10 November. R2-06 consent and unsubscribe before the next patient campaign push.

Hardening: R2-08 to R2-10, R2-12 to R2-14, then CI with scanning (R2-16), so the next review starts from a pipeline rather than a person.

Detailed findings

R2-01	Meta app secret and Google OAuth client secret committed to git	High
Where	`apps/api/src/Acme.WebApi/appsettings.Development.json (tracked; Social:Meta:AppSecret, Social:Google:ClientSecret, Social:Encryption:Keys)`	
Issue	The file holds a real-format Meta app secret (32 hex characters, with its App ID) and a Google OAuth client secret (GOCSPX-…), next to a comment in the same file saying never to put real secrets there. The file is tracked; the values were added in commit 791cb3f (11 September 2026) and are in the history from then on. The first review flagged only the Mailjet key (SEC-03) and missed these.	
Impact	Anyone with read access to the repository, a clone or a backup can act as the app with Meta and Google: exchange authorisation codes, call Graph and Google APIs as the app, and, if the same apps are used in production, impersonate the product in OAuth flows for pharmacies' Facebook, Instagram and Google Business accounts.	
Recommendation	Rotate both secrets now in the Meta developer console and Google Cloud console (rotation is what closes this; deleting the lines alone does not). Move them to user-secrets locally and to App Service settings in production; blank them in the file. Optionally purge them from history with git filter-repo. Add a secret scanner (gitleaks or GitHub secret scanning) as a pre-commit hook and CI step so this cannot recur.	
Status	**Fixed.** Rotated in production on 3 October 2026; production reads both secrets from App Service settings. The previous values remain in appsettings.Development.json as development-only credentials (loaded only in Development; the API warns if a hosted site runs as Development). Secret scanning in CI (.gitleaks.toml) guards every other file.	

R2-02	Next.js 14 is end-of-life and no longer receives security fixes	High
Where	`apps/web/package.json (next 14.2.35, eslint-config-next 14.2.15)`	
Issue	Next.js 14 reached end of life on 26 October 2025. Fixes since then ship only for 15.x and 16.x. Public reporting describes a 13-advisory release in May 2026 and 9 CVEs fixed in July 2026 (16.2.11 / 15.5.21), covering middleware bypass, SSRF, cache poisoning, XSS and denial of service. None of these are backported to 14.2.x.	
Impact	The internet-facing web tier carries known, unpatched vulnerabilities, and will gain more with every release. Which individual CVEs reach this app was not verified one by one; the app uses middleware for page routing, but every BFF route also checks the session cookie, which limits the impact of a middleware bypass.	
Recommendation	Upgrade to a supported line (Next 15.5.x latest, or 16.x), with React to match. Plan a short regression pass: middleware, route handlers, next/headers usage, and the standalone Docker build. Track Next.js security releases from then on.	
Status	**Open.** Not upgraded here: the review environment cannot install packages. docs/UPGRADE_RUNBOOK_2026-10.md lists the exact steps and the files that change.	

R2-03	Per-address rate limits and audit IPs trust a header the caller controls	Medium
Where	`apps/api/src/Acme.WebApi/Program.cs (ForwardedHeaders: KnownNetworks/KnownProxies cleared, ForwardLimit 2); RateLimits.PerAddress; infra/azure/main.bicep (no access restrictions)`	
Issue	The API accepts X-Forwarded-For from any sender. The API and ai-service are each a public App Service with no access restrictions, so a caller can go straight to the API and send a new X-Forwarded-For value on each request. The code comment says nothing authorises on the address, which was true when it was written. Since SEC-05, the sign-in, reset, account-email and screen-pairing limits are all keyed on it. This assumes the App Service front end appends to a client-supplied X-Forwarded-For rather than replacing it; that behaviour was not tested against the live deployment.	
Impact	The SEC-05 per-address limits can be bypassed, which re-opens high-rate credential stuffing (the per-account lock still applies), email flooding through forgot-password (5 per 15 minutes per address becomes unlimited), and unthrottled guessing of screen pairing codes. Audit-log IP addresses can also be forged.	
Recommendation	First confirm it: call the API's sign-in endpoint directly 11 times in a minute, each with a different X-Forwarded-For value; if the eleventh is not refused with 429, the gap is real. Preferred fix: make the web app the only way in. Use App Service access restrictions or VNet integration with private endpoints so the API and ai-service accept traffic only from the web app, then trust only that hop (KnownNetworks set to the web app's outbound range). Alternative: have the BFF sign the client address (HMAC header with a dedicated key) and key the limits on the signed value only.	
Status	**Fixed.** The web app passes only the address its front end recorded, signed with a shared key (CLIENT_IP_SIGNING_KEY / Edge:ClientIpKey). With the key set, the API trusts one hop and believes the address only with a valid signature. Without the key it warns at boot.	

R2-04	Open redirect after sign-in	Medium
Where	`apps/web/src/app/login/page.tsx (sanitizeRedirect)`	
Issue	The check rejects values that start with //, but not /\. Browsers treat a backslash as a slash, so /login?redirect=/%5Cevil.example becomes https://evil.example/ after window.location.assign. Verified with the URL parser.	
Impact	A convincing phishing chain: the victim signs in on the real site and lands on an attacker's page (for example "session expired, re-enter your password").	
Recommendation	Resolve the value with new URL(raw, window.location.origin) and accept it only when the origin is unchanged. Also refuse backslashes and control characters, and return the pathname + search + hash. Add a unit test with /\, /%5C, /\t/ and https: variants.	
Status	**Fixed.** The redirect is resolved as the browser would resolve it and kept only on this origin; backslashes and control characters are refused. Mutation proof run: caught.	

R2-05	Two gaps in the SEC-01/SEC-10 image defences	Medium
Where	`Acme.Application/Posters/CanvasImageSources.cs (regex on "blobKey"); QuestPdfExporter (Deserialize with PropertyNameCaseInsensitive); Acme.Infrastructure/Imaging/SvgRasterizer.cs`	
Issue	(a) The save-time and render-time checks match the literal key "blobKey" in the JSON text, but the renderer reads property names case-insensitively. A shape keyed "BlobKey" or "BLOBKEY" passes both checks and is still drawn. (An escaped key such as "blob\u004bey" passes the save check only: the database stores it unescaped, so the render-time check catches it.) The exporter's own allow-list still stops http fetches and limits raw storage keys to brand/logos/, so this mainly lets a data:image/svg+xml image through. (b) The SVG guard accepts any data: reference, so an SVG can nest a second SVG as data:image/svg+xml, and the inner one is never checked. The SVG library (Svg.Model) contains a WebRequest-based loader, so the nested document's external <image> may be fetched server-side. That last step was not confirmed by running it.	
Impact	Chained, (a) + (b) is a possible route back to the server-side request forgery SEC-01 closed, through the poster renderer. At minimum it bypasses two controls that are meant to be defence in depth.	
Recommendation	Validate the parsed canvas, not the text. Deserialise with the renderer's own options, check every image shape's source, then store the re-serialised canonical JSON. In SvgRasterizer, allow only raster data: URIs (png/jpeg/webp/gif) and refuse nested SVG. Turn off the library's external loading where it is configurable (ResolveExternalImages, ResolveExternalElements, DisableDtdProcessing). Add tests for the case, escape and nesting variants.	
Status	**Fixed.** The canvas is parsed and every source property is checked, whatever its case or escaping; a property named twice is refused. The SVG check allows only raster data: images, refuses CSS escapes, and switches off the library's external loading where its version exposes the switch.	

R2-06	Patient marketing emails have no consent record or unsubscribe	Medium
Where	`Acme.Domain/Patients/Patient.cs; Acme.Application/Campaigns (no consent, opt-out or List-Unsubscribe anywhere in the API)`	
Issue	Pharmacies import patients and email them campaigns. No consent is stored (when, how, for what), campaign emails have no unsubscribe link or List-Unsubscribe header, and there is no suppression list that later campaigns respect. The Terms page tells pharmacies they must honour unsubscribe requests, but the product gives them no way to receive or record one.	
Impact	Under UK GDPR and PECR, marketing email to individuals generally needs consent (or the soft opt-in) and must offer a simple opt-out in every message. Without one, the pharmacies and the platform are exposed to complaints and regulatory action. Missing one-click unsubscribe also hurts deliverability with Gmail and Yahoo bulk-sender rules.	
Recommendation	Add per-patient consent (status, source, timestamp), a signed one-click unsubscribe link and RFC 8058 List-Unsubscribe headers on every campaign email, and an owner-wide suppression list checked at send time. Show consent status in the patient list and import. Take advice on the legal basis; this is not legal advice.	
Status	**Fixed.** Signed one-click unsubscribe (link in every email, RFC 8058 headers) with a public confirm page. A per-pharmacy suppression list is checked when a campaign is created and again at send, and the audience preview shows who was left out. Consent is recorded per patient on the patient form and from an optional Marketing Consent column on import, and is included in exports. Unreadable consent values are refused, never guessed.	

R2-07	Runtimes at or near end of support	Medium
Where	`apps/web/Dockerfile (node:20-alpine), apps/api (net8.0 / aspnet:8.0)`	
Issue	Node.js 20 left support at the end of April 2026, so the web image runs an unsupported runtime. .NET 8 (LTS) support ends on 10 November 2026, about five weeks from this review.	
Impact	After end of support, runtime vulnerabilities are no longer patched, and rebuilding the image stops picking up fixes.	
Recommendation	Move the web image to Node 22 or 24 LTS, ideally together with R2-02. Plan the API's move to .NET 10 (the current LTS) before November, including test projects and the SDK build stage in the Dockerfile.	
Status	**Partly fixed.** Web image on Node 22 LTS. .NET 10 is planned in the upgrade runbook, to be done before 10 November 2026.	

R2-08	The account lock can be used to lock someone else out	Low
Where	`Acme.Infrastructure/Security/MemoryLoginThrottle.cs; AuthService.LoginAsync`	
Issue	The SEC-05 lock is keyed on the email address alone. Eight wrong passwords from anyone lock that account for 15 minutes. Eight attempts fit inside the per-address limit, so a few requests every 15 minutes keep an account locked indefinitely.	
Impact	A known pharmacy or staff address can be kept locked out (targeted denial of service).	
Recommendation	Key the fast lock on account + address and keep a slower account-wide limit. Let a successful password reset clear the lock, and email the account owner when a lock trips.	
Status	**Fixed.** The fast lock is per account and address. The account-wide lock spares addresses that have signed in successfully, and a password reset clears both.	

R2-09	Access tokens outlive deactivation, password reset and sign-out	Low
Where	`JwtTokenService (60-minute access tokens); no per-request status check in the API`	
Issue	Sign-out, password reset and deactivation revoke refresh tokens only. An access token already issued stays valid until it expires, up to 60 minutes.	
Impact	A stolen token, or a just-deactivated user, keeps working for up to an hour.	
Recommendation	Shorten access tokens to 10–15 minutes, or add a security-stamp claim checked in JwtBearerEvents.OnTokenValidated against a short (about 1 minute) cache, and bump the stamp on reset, deactivation and sign-out-everywhere.	
Status	**Fixed.** Each request checks the account can still sign in and has not ended its sessions since the token was issued (30-second cache, cleared at once on the instance making the change). Deactivation and password reset end sessions. Access tokens are now 20 minutes.	

R2-10	Azure resources are reachable from the internet and secrets are not in Key Vault	Low
Where	`infra/azure/main.bicep`	
Issue	Cosmos DB has no network rules (public access, key-based connection string). The container registry has its admin user enabled, and its password is placed in App Settings. JWT, Mongo and storage secrets are plain App Settings rather than Key Vault references. Storage allows shared-key access. FTPS state and SCM access restrictions are not set explicitly.	
Impact	A leaked connection string or key works from anywhere. Secrets are visible to anyone with Website Contributor on the apps, and rotation is manual.	
Recommendation	Restrict Cosmos DB to the App Service outbound IPs or a private endpoint. Use managed identity for ACR pulls (acrUseManagedIdentityCreds) and disable the admin user. Move secrets to Key Vault references. Set ftpsState: 'Disabled' and restrict the SCM site.	
Status	**Partly fixed.** FTPS disabled, TLS 1.2 minimum on app and SCM, FTP publishing credentials off. Still manual (steps in PRODUCTION_CHECKLIST.md §0): Cosmos DB firewall, ACR via managed identity, Key Vault references, and SCM basic auth (kept for az webapp log tail).	

R2-11	ai-service /diagnostics/jwt is anonymous in production	Low
Where	`apps/ai-service/app/main.py (jwt_diagnostics)`	
Issue	Returns a truncated SHA-256 fingerprint of the JWT secret, plus the issuer and audience, to anyone. The API's equivalent answers only in Development, and SEC-08 turned off /docs but not this.	
Impact	Lets an attacker confirm a guessed or leaked secret offline. It is harmless with a strong random secret, but it is an unnecessary oracle.	
Recommendation	Answer only when APP_ENV=development, or only to staff. The smoke check can compare the boot log lines instead.	
Status	**Fixed.** Answers only when APP_ENV=development.	

R2-12	No size or shape limit on saved posters	Low
Where	`PostersController / PosterService (no RequestSizeLimit, no shape cap); CanvasJsonConverter`	
Issue	Saving a poster accepts any body up to the server default (about 30 MB) and any number of shapes, including large inline data: images. The renderer then draws all of it on every export, email image and screen.	
Impact	One account can make exports slow or fail for everyone sharing the instance, and a document over 2 MB fails to save on the Cosmos DB account the Bicep template creates, with an unhelpful error.	
Recommendation	Cap the save body (for example 4 MB), the shape count (for example 500) and inline image size, steering large images to an upload; answer with a clear message.	
Status	**Fixed.** Saves over 1.9 MB or 500 items are refused with a plain message (configurable). An 8 MB request ceiling applies. The editor compresses large pictures before adding them.	

R2-13	One secret signs everything	Low
Where	`Jwt:Secret / JWT_SECRET: access tokens, poster email-image links (180 days), video links, ai-service quota assertions`	
Issue	The same key signs sessions, long-lived public links and service-to-service assertions.	
Impact	Rotating it after an incident breaks every emailed poster image and video link, which discourages rotation. A leak in any one use compromises all of them.	
Recommendation	Derive a separate key per purpose (HKDF from a master secret, or separate settings) and add a key id to links so old links keep verifying during rotation.	
Status	**Fixed.** Emailed poster-image, video and unsubscribe links use per-purpose HKDF keys from Signing:LinkSecret, so the JWT secret can be rotated without breaking them; older links still verify. The two-minute ai-service quota assertion stays on the JWT secret by design: anyone holding that secret can already mint sign-in tokens, so a separate key would add no protection.	

R2-14	ImageMagick security policy loosened in the ai-service image	Low
Where	`apps/ai-service/Dockerfile (policy.xml: @-pattern removed, path rights set to read|write)`	
Issue	The build deletes ImageMagick's @-path restriction and widens path rights for every pattern, to make MoviePy's TextClip work.	
Impact	If untrusted files ever reach ImageMagick, the wider policy restores the file-read and file-write tricks the default policy blocks. Today only caption text reaches ImageMagick, through a temporary file MoviePy writes, and user images go through Pillow, so this is latent rather than exploitable.	
Recommendation	Allow @ reads only for the temp directory MoviePy uses, and explicitly disable the risky coders (URL, HTTP/HTTPS, MVG, MSL, EPHEMERAL).	
Status	**Fixed.** URL, HTTP(S), FTP, MVG, MSL and EPHEMERAL coders are disabled in the image's ImageMagick policy.	

R2-15	Personal and possibly health data handling	Info
Where	`Patient.Notes (free text); information-level logs include account emails`	
Issue	Pharmacists can type anything into patient notes, which in practice may include health information (special-category data). Information logs record account emails.	
Impact	Raises the data-protection bar: a DPIA, retention rules, and care with log access and retention.	
Recommendation	Add guidance on the notes field (or remove it), set a retention policy, record a DPIA, and redact or hash emails in logs.	
Status	**Fixed.** Email addresses in logs are masked (j•••e@example.com) and phone numbers cut to their last three digits, including the mock email and SMS senders that also run on the test environment. The notes field says not to record health details. docs/DATA_PROTECTION.md records what is held, where and for how long, with a DPIA outline. Log-sink retention (30 days or less) is a hosting setting.	

R2-16	Supply-chain and process gaps	Info
Where	`repository root (no CI workflows, no secret scanning, no dependency audit)`	
Issue	There is no CI, so the test suites, check scripts, dependency audit and secret scan run only when someone remembers. python-jose is lightly maintained; Svg.Skia 1.0.0.18 and MoviePy 1.0.3 are old. The 12 test-report files are still tracked (SEC-16 follow-up).	
Impact	Regressions and leaked secrets (as in R2-01) are found by people, late.	
Recommendation	Add a CI pipeline: pnpm test, pnpm audit, dotnet list package --vulnerable, pip-audit and gitleaks on every push. Consider PyJWT instead of python-jose. Run git rm -r --cached playwright-report test-results .qa*.	
Status	**Fixed.** GitHub Actions CI runs every suite and check, dotnet list package --vulnerable, pip-audit and gitleaks. pnpm audit reports but does not block until the Next.js upgrade (R2-02) lands, or CI would stay red. Tracked report files still need git rm --cached.	

What is done well

• Every endpoint carries an explicit authorisation attribute, at class or action level; anonymous endpoints are a short, deliberate list (sign-in flows, public template gallery and assets, signed email-image and video links, kiosk, Stripe webhook with signature check, invitation preview).

• "Manage as" delegation is tightly scoped: own children only, no chaining, active accounts only, 30-minute tokens, and the person (not the account) recorded in the audit log.

• JWTs validated for issuer, audience, lifetime and key in both services (HS256 pinned in ai-service); cookies httpOnly, SameSite=Lax and Secure in production.

• Signed links use HMAC with constant-time comparison and expiry; Mongo regex input is escaped; no dangerouslySetInnerHTML, eval or shell-string subprocess calls; video clip URLs are worker-owned, not user-supplied.

• The first-round fixes are in place and covered by unit, integration and ai-service tests plus mutation proofs (pnpm test:prove -- SEC).

• No live keys for Stripe, Anthropic, Replicate, Tavily, Pexels, ElevenLabs, AWS or Azure Storage in tracked files (pattern scan of 916 tracked and unignored files); storage blocks public blob access and enforces TLS 1.2.

Method and limits

Reviewed: every controller's authorisation attributes and the anonymous surface; the BFF routes, middleware, cookies and redirects; the ai-service routes, JWT handling and outbound fetches; the first-round fixes, looking for bypasses; secrets in all tracked and unignored files (pattern scan) and in every appsettings file; dependency and runtime versions (Next.js support status checked against current public sources); Dockerfiles; and the Azure Bicep template.

Not covered: live penetration testing, the real App Service and Azure settings, and per-CVE applicability for R2-02. The .NET and ai-service endpoint tests could not be run in the review environment; run pnpm test:api, the integration suite and pnpm test:ai to confirm the first-round fixes pass. R2-05(b) depends on the SVG library's runtime behaviour and should be confirmed with a test before or alongside the fix. Nothing in this report is legal advice; R2-06 should be checked with a data-protection adviser.

Describes the repository as of 3 October 2026, including uncommitted work in the working tree.